WEB SERVER SECURITY

Cross-Site Request Forgery (CSRF)

GET-Based Forged Action: Exploitation and Mitigation

Institution	Indian Institute of Information Technology Allahabad
Department	Department of Information Technology
Course	Cyber Security (IT-30001)
Semester	Autumn Semester 2026
Student	Vraj Chalodiya
Roll number	IIT2024241
Vulnerability	Cross-Site Request Forgery (CSRF)
Variant	GET-based forged action
Implementation	JavaScript, Node.js and Express

Practical report with implementation details and screenshot evidence

1. Introduction

Cross-Site Request Forgery (CSRF) is a web application vulnerability in which an attacker causes a victim's browser to send an unwanted request to a website where the victim is already authenticated. The browser may automatically attach the victim's session cookie, so the target application can mistake the forged request for an intentional action by the user.

This practical demonstrates a GET-based forged action against a deliberately vulnerable local account application. The action changes the signed-in user's email address. It then demonstrates the mitigation: when protection is enabled, the same forged GET request is rejected with HTTP 403. The application also contains a protected POST email-update route that requires a session-bound synchronizer token.

2. Aim and objectives

• Build and run a small local web application using JavaScript, Node.js, Express and express-session.
• Demonstrate a GET request that changes account data without proof of user intent.
• Trigger that request from a separate untrusted page using an automatically loaded image.
• Enable the mitigation and repeat the same request to show that it is blocked.
• Demonstrate a legitimate POST update protected by a synchronizer CSRF token.
• Explain the root cause and the other CSRF mitigation options listed in the assignment.

3. Tools and environment

Language: JavaScript (Node.js). Web framework: Express. Session handling: express-session. Browser: Google Chrome. Local application: http://127.0.0.1:5000. Untrusted/attacker demonstration page: http://127.0.0.1:5001. All testing was performed against the student's own local demonstration application.

Demo login credentials used by the application: username student; password csrf-demo. These are intentionally simple demonstration credentials and must not be reused in a real application.

4. Application design

The project uses two local Express servers. The main application maintains the authenticated session, displays the current email address, exposes the vulnerable GET route, and provides a protected POST route. The second server hosts an untrusted page that contains an image element pointing to the vulnerable GET route on the main application.

The main application stores the signed-in user, current email, mitigation flag and CSRF token in the session. The session cookie is configured as HttpOnly and SameSite=Lax in this local demonstration. The code uses a placeholder session secret and the default in-memory session store; these are for classroom demonstration only, not production deployment.

5. Vulnerable implementation and root cause

The vulnerable route is GET /vulnerable/change-email. When mitigation is disabled, it reads the email query parameter, validates its basic email format, and updates the email stored in the authenticated session. The route does not require a CSRF token or another proof that the user intended the change.

Root cause: the application performs a state-changing operation in response to a GET request and relies only on the browser's automatically attached session cookie. A third-party page can cause the victim's browser to issue that request while the victim is signed in.

6. Exact exploit payload and execution

The attacker-controlled page includes an image whose source is the following URL. Loading the page makes the browser request the URL automatically; the image itself does not need to be visible.

GET http://127.0.0.1:5000/vulnerable/change-email?email=changed-by-csrf%40example.test

The browser sends the request to the main application. If the user has an active session and mitigation is disabled, the application updates the email to changed-by-csrf@example.test. The exploit relies on the browser attaching the session cookie to the request. The two local servers use different ports, but the request targets the same 127.0.0.1 host as the authenticated application.

7. Mitigation implemented and verification

The demonstration includes a mitigation switch on the dashboard. When enabled, the vulnerable GET handler refuses to perform the state change and returns HTTP 403 Forbidden. The exact same attacker page and GET payload were then loaded again. The browser displayed “403 Forbidden — CSRF mitigation blocked this forged GET request.” This is the blocked-attempt evidence.

Security design note: the strongest design is not to expose state-changing functionality through GET at all. The GET endpoint in this teaching application is retained only so that the before/after behavior can be demonstrated. In a real application, account changes should use a state-changing method such as POST and require CSRF protection. The application’s protected POST route demonstrates that approach using a session-bound synchronizer token.

8. Protected POST implementation

The dashboard contains a form that submits the new email to POST /secure/change-email. The server generates a random token when the user logs in, stores it in the session, and places it in the legitimate form as a hidden field. On submission, the server compares the submitted token with the session token using crypto.timingSafeEqual. A missing or incorrect token results in HTTP 403; only a valid token proceeds to email validation and update.

Relevant implementation logic (simplified from app.js):

```
app.post('/secure/change-email', login, (req, res) => {
  const submitted = Buffer.from(String(req.body.token || ''));
  const expected = Buffer.from(String(req.session.token || ''));

  if (submitted.length !== expected.length ||
      !crypto.timingSafeEqual(submitted, expected)) {
    return res.status(403).send('403 Forbidden: invalid CSRF token');
  }

  const email = String(req.body.email || '');
  if (!validEmail(email)) return res.status(400).send('Invalid email');

  req.session.email = email;
  res.redirect('/dashboard');
});
```

9. Other CSRF mitigations (theory)

Synchronizer anti-CSRF tokens: The server generates an unpredictable token, associates it with the user’s session, and requires it on state-changing requests. An attacker on another site should not be able to read the token, so the forged request cannot supply the correct value. This project demonstrates this technique on the protected POST route.

SameSite cookies: The SameSite attribute instructs the browser when to include cookies on cross-site requests. Strict provides the strongest cross-site restriction; Lax allows cookies in some top-level navigations; None permits cross-site cookie use and requires Secure. SameSite is useful defense in depth, but should not be treated as the only protection for every application flow.

Re-authentication for sensitive actions: Before especially sensitive operations, the application asks the user to re-enter a password or complete another authentication step. A forged request cannot normally complete that interactive confirmation by itself. This is appropriate for actions such as changing credentials, transferring funds or changing account recovery settings.

Avoid state-changing GET requests: GET should be safe and should retrieve information rather than modify server-side state. Put changes behind POST/PUT/PATCH/DELETE as appropriate and protect them with CSRF tokens and suitable cookie settings. This prevents simple cross-site resource loads, such as images and links, from changing account data.

10. Results and observations

1. Initial state: the dashboard showed student@example.test and mitigation DISABLED.
2. Exploit page: the untrusted page displayed a prize-themed message and automatically attempted the forged GET action.
3. Successful attack: after visiting the attacker page with protection disabled, the dashboard showed changed-by-csrf@example.test.
4. Mitigation enabled: the dashboard displayed mitigation ENABLED.
5. Blocked replay: repeating the same forged GET URL returned HTTP 403 Forbidden.
6. Protected legitimate update: the dashboard later showed vraj@example.test after a valid protected POST form submission.

These observations demonstrate both the vulnerable behavior and the mitigation behavior in the local application. The protected POST screenshot shows a successful legitimate update; it is not a screenshot of an attacker's forged POST request.

11. Screenshot evidence

Figure 1 shows the initial authenticated dashboard before the attack. Figure 2 shows the untrusted attacker-controlled page. Figure 3 shows the email changed after the GET-based CSRF attempt. Figure 4 shows mitigation enabled. Figure 5 shows the same forged GET request blocked with HTTP 403. Figure 6 shows a legitimate protected POST email update.

12. Conclusion

The experiment demonstrates how a state-changing GET endpoint can be abused through CSRF when the application accepts the browser's session cookie as sufficient evidence of user intent. The attacker page caused the browser to send the forged request, and the vulnerable application changed the account email. After enabling the mitigation, the same request was rejected with HTTP 403. A separate protected POST route demonstrated session-bound synchronizer-token validation for legitimate updates.

The key lesson is to avoid state changes through GET, validate CSRF tokens on state-changing requests, configure cookies carefully, and require re-authentication for highly sensitive actions. Testing was limited to the locally hosted demonstration application.

13. Individual ownership declaration

Member name: Vraj Chalodiya
Roll number: IIT2024241
Category: Cross-Site Request Forgery (CSRF)
Variant implemented: GET-based forged action
Additional demonstration: synchronizer-token-protected POST email update

14. Run instructions

From the project root, install dependencies with npm install. Start the main application in one terminal using npm run main. Open http://127.0.0.1:5000/login and sign in with the demo credentials. Start the attacker server in a second terminal using npm run attacker. Open http://127.0.0.1:5001/ to trigger the forged GET request. Return to the dashboard to inspect the email. Enable CSRF mitigation and revisit the attacker page to observe the 403 response. Use the protected POST form on the dashboard to demonstrate a legitimate token-validated update.

Use 127.0.0.1 consistently for both applications; do not mix it with localhost during the demonstration because browser cookies are host-scoped.

15. Limitations and production considerations

This is an educational localhost demonstration, not a production-ready account system. The credentials and session secret are hard-coded for the lab, the session store is in-memory, HTTPS is not configured, and the vulnerable GET route is intentionally retained behind a demonstration toggle. A production system should use strong secret management, HTTPS, a production session store, secure cookie settings, robust input validation, appropriate authorization checks, and CSRF protection on every relevant state-changing request.

Appendix A — Screenshot evidence

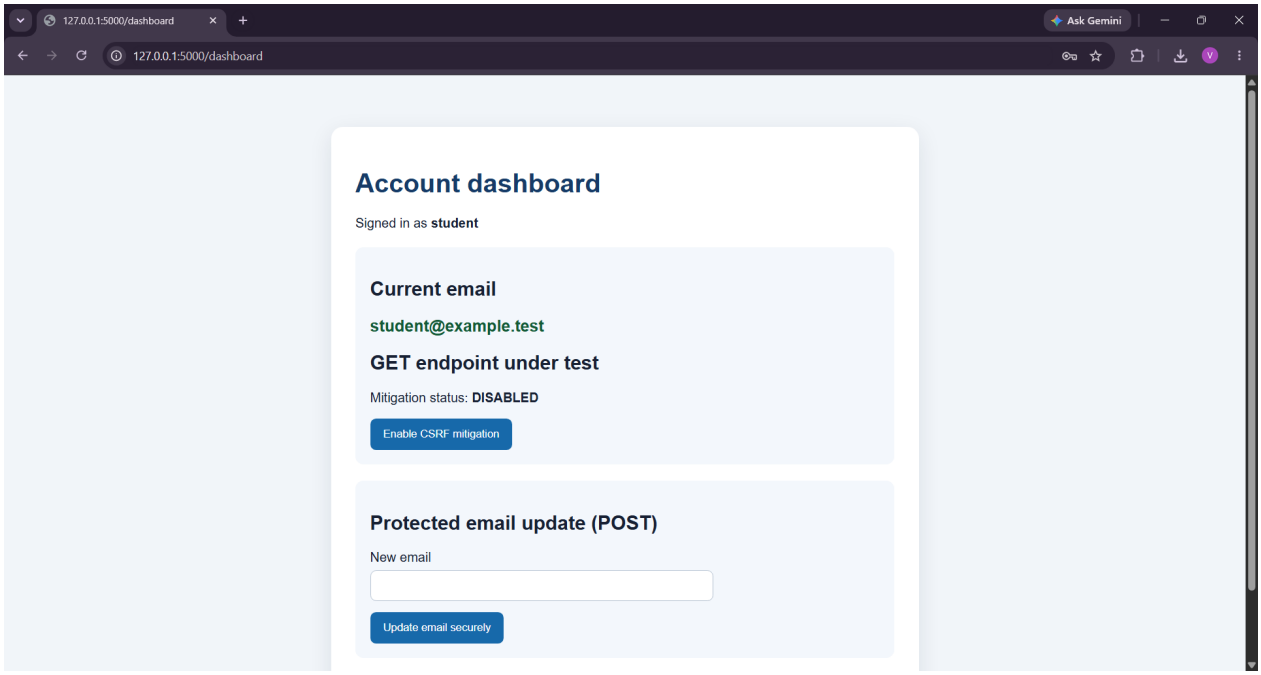


Figure 1. Initial authenticated account dashboard. The current email is student@example.test and CSRF mitigation is disabled.

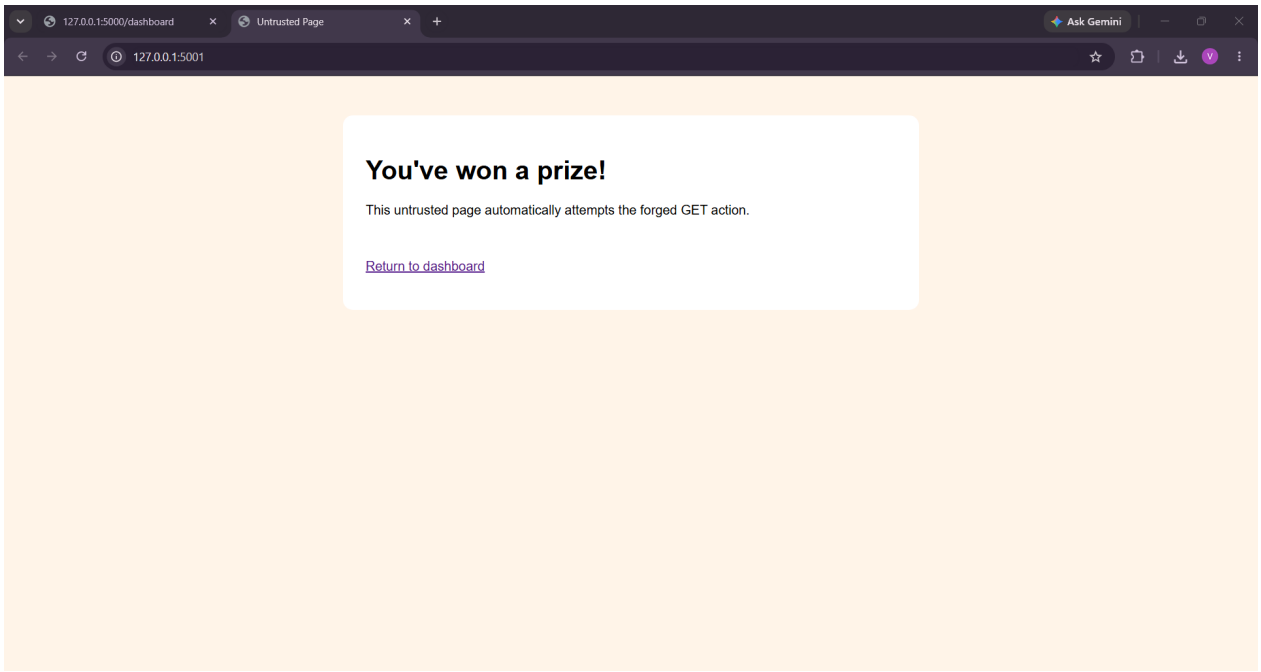


Figure 2. Untrusted attacker-controlled page. Loading this page automatically requests the vulnerable GET URL through an image element.

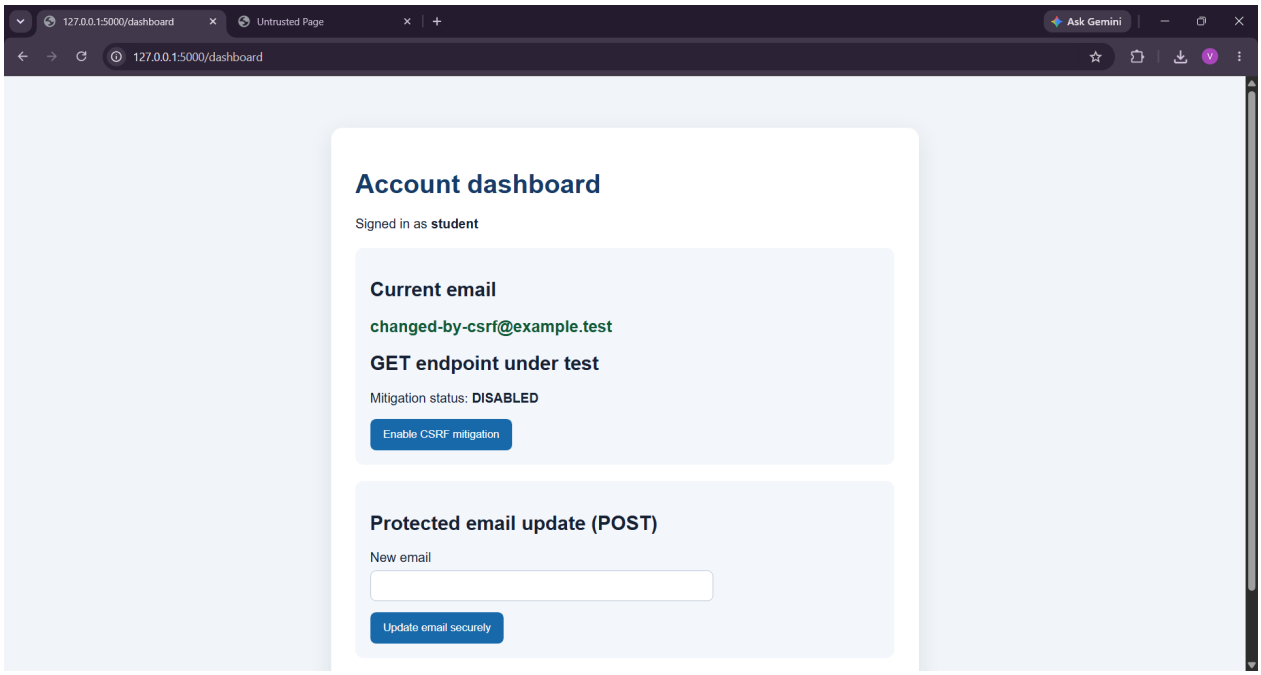


Figure 3. Successful GET-based CSRF attack. The dashboard shows the email changed to changed-by-csrf@example.test.

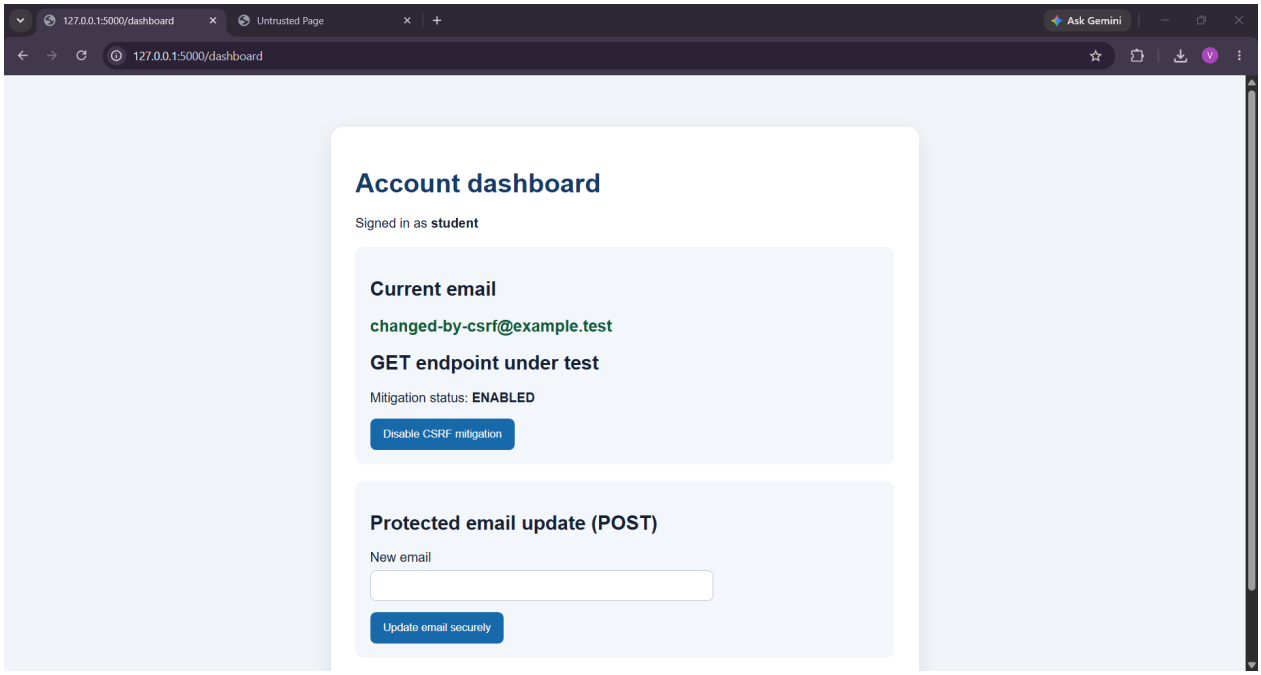


Figure 4. CSRF mitigation enabled on the account dashboard.

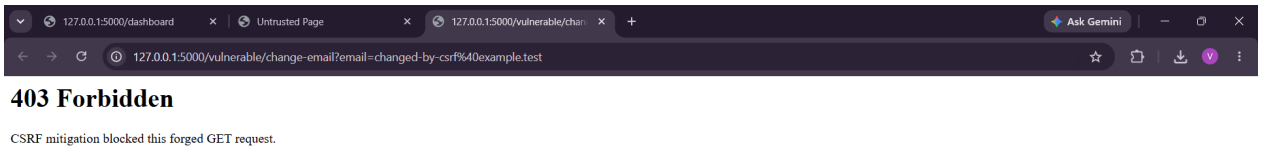


Figure 5. The same forged GET request is blocked. The server returns HTTP 403 Forbidden.

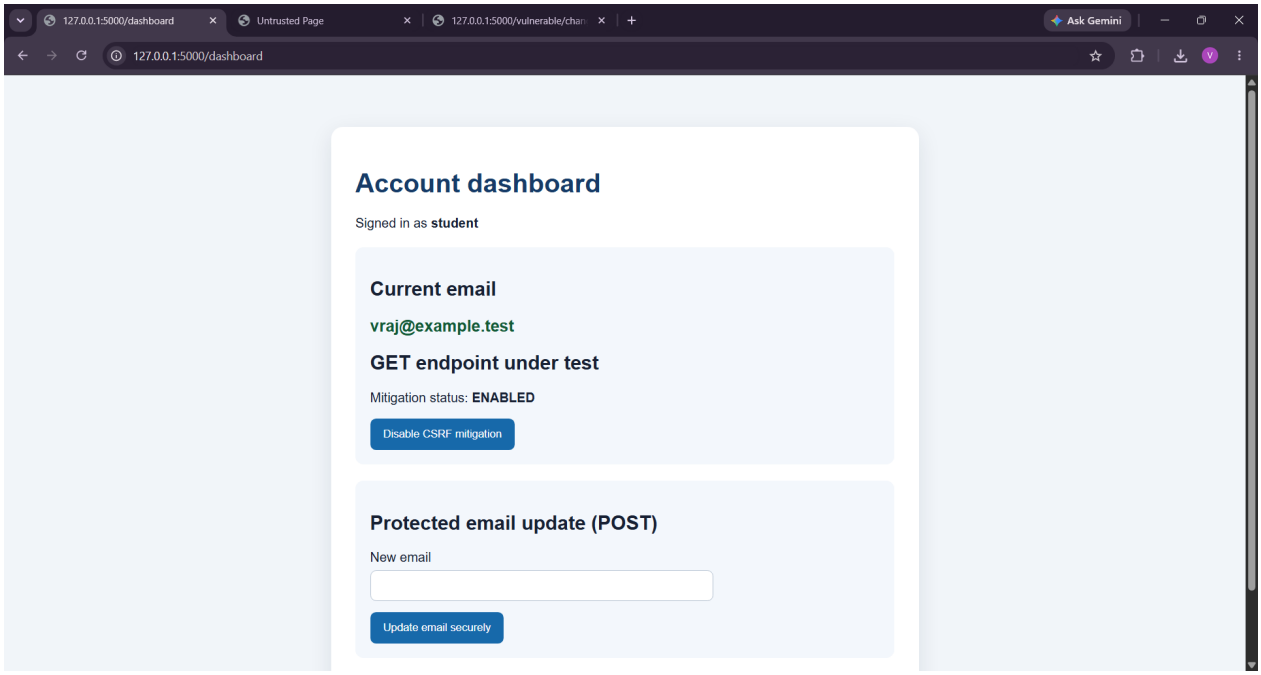


Figure 6. Successful legitimate email update through the protected POST form. The dashboard shows vraj@example.test.